Project Report

On

HOSPITAL MANAGEMENT SYSTEM

Submitted in partial fulfillment of the
requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

SUBMITTED BY

KAKI VARAPRASAD

(22A91F0019)

Under the Esteemed Guidance of

Mr K.C.PRADEEP, M.Tech

Associate Professor

DEPARTMENT OF MCA

ADITYA ENGINEERING COLLEGE

An Autonomous Institution

(Approved by AICTE, Affiliated to JNTUK & Accredited by NBA, NAAC with 'A++' Grade)

Recognized by UGC under the sections 2(f) and 12(B) of the UGC act 1956

SURAMPALEM- 533 437, E.G. Dt, ANDHRA PRADESH

2022-2024

ADITYA ENGINEERING COLLEGE

An Autonomous Institution

(Approved by AICTE, Affiliated to JNTUK & Accredited by NBA, NAAC with 'A++' Grade)Recognized by UGC under the sections 2(f) and 12(B) of the UGC act 1956

SURAMPALEM- 533 437, E.G. Dt, ANDHRA PRADESH

DEPARTMENT OF MCA

CERTIFICATE

This is to certify that the project work entitled, **"HOSPITALMANAGEMENT SYSTEM",** is a bonafide work carried out by **K.VARAPRASAD** bearing Regd.No:**22A91F0019** submitted to the requirements for the award of the Computer Applications in partial fulfilment of the requirements for the award **o**f degree of **MASTER OF COMPUTER APPLICATIONS** from **Aditya Engineering College**, Surampalem duringthe academic year **2022-2024**.

Project Guide
Mr. K.C. PRADEEP., M. Tech
Associate Professor,
Department of MCA,
Aditya Engineering College,
Surampalem-533437.

Head of the Department
Mrs. D. Beulah., M. Tech., (Ph. D)
Associate Professor,
Department of MCA,
Aditya Engineering College,
Surampalem-533437.

EXTERNAL EXAMINER

DECLARATION

I here declare that the project entitled **"HOSPITAL MANAGEMENT SYSTEM",** to **Aditya Engineering College, Surampalem** has been carried out by mealoneunder the guidance of **Mr. K.C. PRADEEP.**

Place: Surampalem

Date: K.VARAPRASAD

(22A91F0019)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crownedour efforts with success. It is a pleasant aspect that I have now the opportunityto express my gratitude for all of them.

The first person I would like to thank is my guide **Mr. K.C. Pradeep , M.Tech Associate Professor, Department of MCA**, **Aditya Engineering College, Surampalem.**His wide knowledge and logical way of thinking havemade a deep impression on me. His understanding, encouragement and personal guidance have provided the basis for this project. He is a source of inspiration for innovative ideas and his kind support is well knownto all his students and colleagues.

I wish to thank **Mrs. D. Beulah, M. Tech, (Ph. D), Head of the Department, Aditya Engineering College, Surampalem,** who has extended her support for the success of this project.

I also wish to thank **Sri. M. Sreenivasa Reddy, Principal, Aditya Engineering College, Surampalem,** who has extended his support for the success of this project.

I would like to thank all the **Management & Technical SupportingStaff** for theirtimely help throughout the project.

ABSTRACT

The Hospital Management System (HMS) is a comprehensive software solution designed to streamline and enhance the efficiency of healthcare institutions by integrating and automating various administrative, clinical, and financial processes. This system aims to improve the overall quality of patient care, optimize resource utilization, and facilitate seamless communication among healthcare professionals. The HMS includes modules for patient registration, appointment scheduling, electronic health records (EHR), billing and invoicing, inventory management, and staff administration.

By centralizing these functions, the system reduces paperwork, minimizes errors, and enables real-time access to critical patient information, ultimately leading to faster decision-making and better healthcare outcomes. With user-friendly interfaces and robust security features, the Hospital Management System serves as a powerful tool to modernize healthcare administration, fostering a more efficient and patient-centric approach to hospital operations.

Furthermore, the HMS incorporates advanced analytics and reporting tools, allowing administrators to gain valuable insights into hospital performance, resource utilization, and patient outcomes. This data-driven approach enables evidence-based decision-making, helping hospitals to identify trends, optimize workflows, and implement strategic improvements.

CONTENTS

LIST OF FIGURES

CHAPTER-1
INTRODUCTION

1. INTRODUCTION

1.1 Brief Information about the Project:

In healthcare, an efficient and integrated management system is crucial for delivering quality patient care, optimizing operational processes, and ensuring the overall effectiveness of healthcare institutions.

The Hospital Management System is a comprehensive software solution that serves as a centralized hub for managing diverse aspects of hospital operations, ranging from patient registration and appointment scheduling to medical records, billing, and inventory management.

This system is designed to replace traditional, paper-based methods with streamlined and automated processes, aiming to enhance the overall efficiency and effectiveness of healthcare delivery.

The Hospital Management System not only facilitates the smooth flow of information among various departments within a hospital but also contributes to improved decision-making by providing real-time access to critical data.

As hospitals and healthcare providers strive to meet the increasing demands of a growing population, the implementation of a robust Hospital Management System becomes imperative for ensuring optimal resource utilization, reducing errors, and ultimately elevating the standard of patient care.

In this rapidly evolving landscape, the Hospital Management System emerges as a cornerstone in the modernization of healthcare administration, promising a more interconnected, responsive, and patient-centric approach to hospital management. The Hospital Management System not only facilitates the smooth flow of information among various departments within a hospital but also contributes to improved decision-making by providing real-time access to critical data.

robust Hospital Management System becomes imperative for ensuring optimal resource utilization, reducing errors, and ultimately elevating the standard of patient care.

1.2 Motivation and Contribution of the Project:

The development and implementation of the Hospital Management System are driven by several key motivations and the anticipation of significant contributions to the realm of vehiclefleet management. Understanding these factors is crucial for appreciating the project's goals andthe positive impact it aims to make.

1. Enhanced Operational Efficiency:

Motivation*:* Traditional methods of managing a fleet of vehicles often involve cumbersome paperwork, manual data entry, and time-consuming processes. This can lead to operational inefficiencies, delayed decision-making, and increased downtime for vehicles.

Contribution: The Automobile Management System seeks to enhance operational efficiency by automating routine administrative tasks. Automation allows for real-time tracking, predictive maintenance scheduling, and streamlined processes, leading to a more agile and responsive fleet management approach.

2. Proactive Maintenance Strategies:

Motivation: Traditional fleet management practices may rely on reactive maintenance, addressing issues only when they become apparent. This reactive approach can result in unexpected breakdowns, higher maintenance costs, and disruptions to operations.

Contribution: The project aims to contribute to proactive maintenance strategies by leveragingreal-time data and predictive analytics. This shift allows organizations to anticipate maintenanceneeds, schedule preventive measures, and reduce the likelihood of unexpected vehicle failures,ultimately minimizing downtime and associated costs.

3. Improved Accessibility to Information:

Motivation: Access to timely and accurate information about vehicle details, maintenance history, and usage patterns is crucial for informed decision-making. Traditional methods may limit accessibility and hinder the quick retrieval of data.

Contribution: The Automobile Management System addresses this motivation by providing a centralized digital platform that allows fleet managers easy and immediate access to comprehensive information. This contributes to faster decision-making, improved resource allocation, and better overall management of the vehicle fleet.

1.3 Objective of the project:

The Hospital Management System project is driven by a set of clear and overarching objectives, each designed to address specific challenges in traditional fleet management and contribute to the improvement of overall operational efficiency and effectiveness. The primaryobjectives include:

1. Implementation of Predictive Maintenance Strategies:

- *Objective:* Integrate predictive maintenance capabilities to identify potential issues before they lead to vehicle breakdowns.
- *Rationale:* Proactive maintenance scheduling based on real-time data helps prevent unexpected failures, reduce downtime, and optimize maintenance costs.

2. Enhancement of Operational Efficiency:

- *Objective:* Streamline administrative tasks and overall fleet management processes for improved efficiency.
- *Rationale:* The project aims to eliminate inefficiencies associated with manual tracking, paperwork, and reactive decision-making, leading to a more agile andresponsive operational model.

3. Improved Accessibility to Comprehensive Information:

- Objective: Provide a centralized digital platform for easy and immediate access to comprehensive information about vehicle details and maintenance history.
- Rationale*:* Enhanced accessibility ensures that fleet managers can quickly retrieve accurate information, enabling timely decision-making and resource allocation.

1.4 Organization of the project:

The implementation of the Hospital Management System project follows a structured and organized approach, encompassing key phases to ensure its successful development and deployment.

- **Project Initiation:**
- Define project goals, objectives, and scope.
- Identify stakeholders and establish communication channels.
- Formulate a project team with assigned roles and responsibilities.
- **Requirement Analysis:**
- Conduct a comprehensive analysis of the requirements for the Auto Management System.
- Collaborate with end-users and stakeholders to gather input and expectations.
- Develop a detailed requirements document outlining system functionalities.
- **System Design:**
- Create a system architecture and design based on the gathered requirements.
- Define the database structure, user interfaces, and integration points.
- Develop wireframes and prototypes to visualize the system's layout and functionality.
- **Development:**
- Implement the designed system architecture and functionalities.
- Integrate necessary technologies, such as GPS, telematics, and security features.
- Conduct regular testing and debugging to ensure the system's reliability.
- **Testing and Quality Assurance:**
- Perform rigorous testing, including unit testing, integration testing, and useracceptance testing.
- Identify and address any bugs, glitches, or performance issues.
- Validate that the system meets the specified requirements and user expectations.

Conclusion:

In conclusion, the Hospital Management System stands as a transformative solution, enhancing healthcare administration by promoting efficiency, accuracy, and patient-centric care.

These systems aim to streamline hospital operations, enhance communication between healthcare professionals, and improve patient care through efficient information management.It's important to note that the field of healthcare informatics is dynamic, and new systems may have been developed or existing ones updated since my last update. Additionally, the adoption of specific systems can vary among healthcare institutions based on factors such as size, budget, and specific needs. For the most current and detailed information, it is recommended to check with healthcare organizations or consult recent reviews and industry reports.

CHAPTER-2
SYSTEM ANALYSIS

2.SYSTEM ANALYSIS

2.1.EXISTING SYSTEM

These systems typically include modules for patient registration, appointment scheduling, electronic health records (EHR), billing, inventory management, and more. Some well-known HMS software includes Epic Systems, Cerner, Allscripts, and Meditech, among others. These systems aim to streamline hospital operations, enhance communication between healthcare professionals, and improve patient care through efficient information management.It's important to note that the field of healthcare informatics is dynamic, and new systems may have been developed or existing ones updated since my last update. Additionally, the adoption of specific systems can vary among healthcare institutions based on factors such as size, budget, and specific needs. For the most current and detailed information, it is recommended to check with healthcare organizations or consult recent reviews and industry reports.

2.2Proposed System:

A proposed Hospital Management System (HMS) envisions an integrated and technologically advanced solution to enhance the overall efficiency and effectiveness of healthcare administration. The system aims to address existing challenges in hospital management by incorporating modern technologies and streamlined processes. Key features of the proposed system may include:

1. **Centralized Information Hub:**
   - Establish a centralized database for seamless management of patient records, appointments, and medical histories, ensuring quick and secure access for healthcare professionals.
2. **Appointment Scheduling and Patient Registration:**
   - Integrate an intuitive appointment scheduling system that allows patients to book appointments online, reducing wait times and optimizing resource allocation.
   - Streamline patient registration processes to capture and manage demographic information efficiently.
3. **Electronic Health Records (EHR):**
   - Implement a comprehensive EHR system for real-time tracking and updating of patient information, ensuring healthcare providers have instant access to accurate and up-to-date data.
4. **Billing and Financial Management:**
   - Develop a robust billing module that automates invoicing, insurance claims, and financial transactions, minimizing errors and improving revenue cycle management.
5. **Inventory and Resource Management:**
   - Incorporate an inventory management system to monitor and manage medical supplies, equipment, and pharmaceuticals efficiently, preventing shortages or overstock situations.
6. **Reporting and Analytics:**
   - Integrate advanced reporting and analytics tools to provide hospital administrators with insights into key performance indicators, enabling data-driven decision-making for strategic planning and resource optimization.
7. **User-Friendly Interfaces:**
   - Design user-friendly interfaces for both healthcare professionals and

2.3Feasibility study:

A feasibility study is a crucial step in assessing the viability and potential success of the proposed

Automobile Management System. It encompasses various aspects to determinewhether the project is financially, technically, and operationally feasible. Here's an overview ofthe feasibility study for the proposed system:

- **Technical Feasibility:**
- Assessment of Technology Requirements*:* Evaluate the technical infrastructure neededfor system development, including software, hardware, and integration capabilities.
- Expertise and Skill Availability: Determine if the required technical skills for system development and maintenance are available within the project team or if external expertise is necessary

Operational Feasibility:

- Impact on Current Operations*:* Analyz how the proposed system will integrate into

- Training Needs*:* Assess the training requirements for end-users and administrators to ensure a smooth transition to the new system.
- **Economic Feasibility:**
- Cost-Benefit Analysis*:* Conduct a comprehensive cost-benefit analysis, comparing theinitial setup costs, ongoing maintenance expenses, and potential long-term savings with the expected benefits and returns on investment.
- Return on Investment (ROI)*:* Evaluate the projected ROI to determine if the financialgains justify the initial investment in the Automobile Management System.
- **Legal and Regulatory Feasibility:**
- Compliance with Regulations*:* Ensure that the proposed system adheres to legal and regulatory requirements related to data protection, privacy, and any industry-specific regulations.
- Risk Assessment*:* Identify and assess legal risks associated with the implementation ofthe system and establish mitigation strategies.
- **Scheduling and Time Feasibility:**
- Project Timeline*:* Develop a realistic project timeline, considering the time required for system development, testing, deployment, and training.
- Dependencies and Constraints*:* Identify potential dependencies and constraints that may impact the project schedule and explore ways to mitigate risks.

2.4Functional Requirements:

Functional requirements outline the specific features, capabilities, and interactions that the proposed Automobile Management System must have to meet the needs of users and achieve its objectives. Here are key functional requirements for the system:

- **User Authentication and Authorization:**
- *Requirement:* Implement a secure user authentication mechanism to ensure that only authorized personnel can access the system.
- *Rationale:* Enhances system security by preventing unauthorized access andmaintaining data integrity.
- **Vehicle Registration and Profile Management:**
- *Requirement:* Allow users to register new vehicles into the system, providing detailssuch as make, model, registration number, and ownership information.

2.5Non-Functional Requirements:

Non-functional requirements describe the characteristics and constraints of the proposed Automobile Management System that are not related to specific functionalities but areessential for its overall performance, usability, and security. Here are key non-functional requirements for the system:

- **Performance:**
- *Requirement:* The system should support simultaneous tracking and management of a

minimum of 500 vehicles without performance degradation.

- *Rationale:* Ensures the system's responsiveness and efficiency under varyingworkloads.
- **Scalability:**
- *Requirement:* The system architecture must be scalable to accommodate an increase inthe number of vehicles without significant changes to performance.
- *Rationale:* Allows the system to adapt to the organization's growing fleet size.
- **Reliability:**
- Requirement : The system should have an uptime of at least 99.9%, ensuring its availability for users at all times.

2.6 System Requirements:

System requirements for an Hospital Management System (HMS) are critical forthe effective functioning of the software in managing and monitoring various aspects of a fleetof vehicles. These requirements encompass hardware, software, network, and security specifications. Here's an outline of the system requirements for an AMS.

Network Requirements:

1. High-speed internet connection for server hosting and data transmission.
2. Firewalls and network security measures to protect against unauthorized access.
3. Virtual Private Network (VPN) support for secure remote access.

Database Requirements:

1. Database Management System: MySQL 8.0 or later.
2. Adequate storage space for storing vehicle data, maintenance records, and system logs.

2.6.1Software Requirements:

- Backend : Python, Django, MySql
- Front-end Technologies : HTML, CSS

2.6.2Hardware Requirements:

- Operating system : Windows 11
- Processor : RYZEN 5
- ROM : 512 GB
- RAM : 8GB

CHAPTER-3

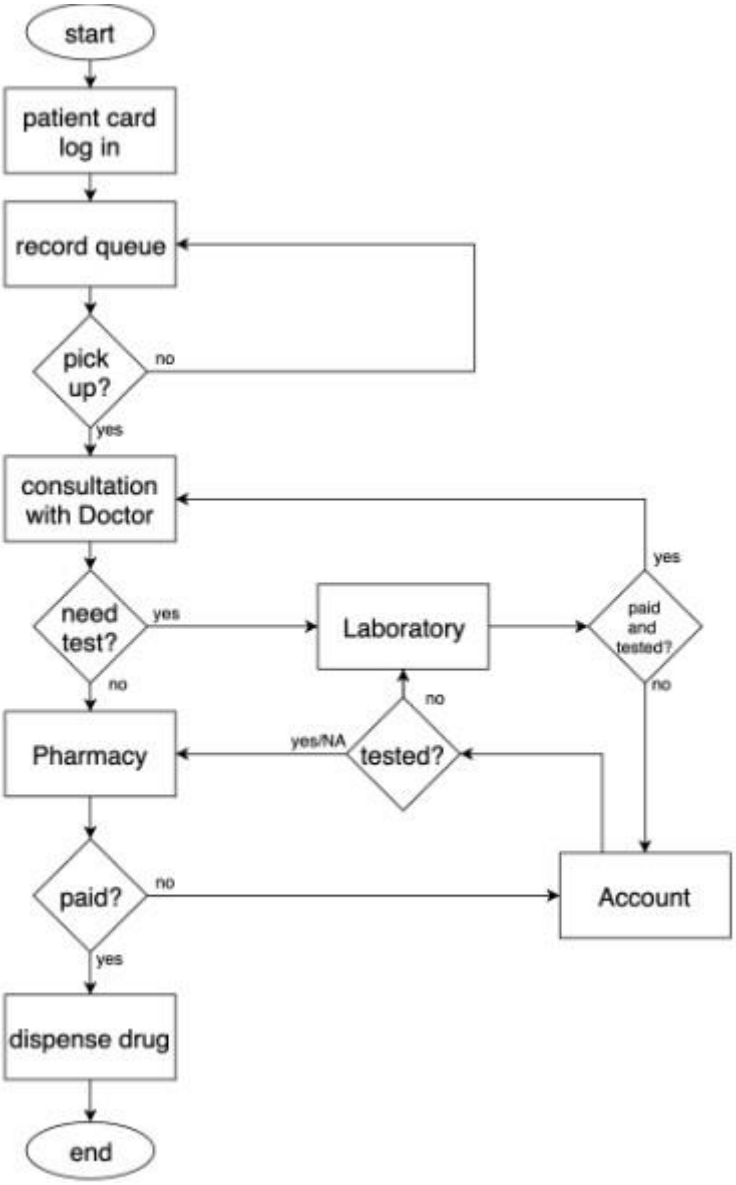
SYSTEM DESIGN

3.SYSTEM DESIGN

3.1 INTRODUCTION

System design is a crucial phase in the software development life cycle, where theoverall architecture and structure of a system are conceptualized. It involves converting the specified requirements into a blueprint that guides the implementation process. Through carefulconsideration of user needs, hardware, and software constraints, system design aims to create arobust and scalable solution. It encompasses both high-level-design, defining system architecture, and low-level design, specifying individual components and their interactions. Effective system design is foundational for building reliable, efficient, and maintainable software systems

3.2 System Architecture:



3.3UML DIAGRAMS

3.3.1Use Case Diagram:

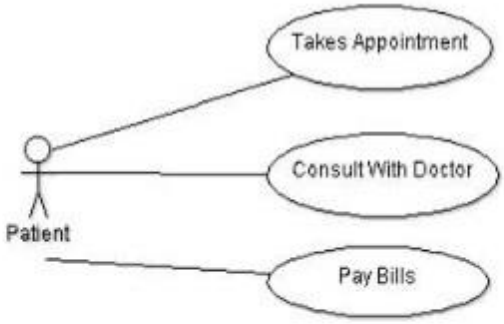
A use case diagram for a Hospital Management System (HMS) represents the interactions between users (actors) and the system to accomplish specific tasks or functionalities. Here's a basic example of a use case diagram for a Hospital Management System:

1. Manage Patient Information:
   - Actors: Admin, Doctor, Nurse
   - Functionality: Add, update, or delete patient information.
2. Schedule Appointment:
   - Actors: Receptionist, Patient
   - Functionality: Schedule, modify, or cancel patient appointments.
3. Diagnose and Prescribe:
   - Actors: Doctor
   - Functionality: Diagnose illnesses, prescribe medications, and update patient records.
4. Manage Patient Care:
   - Actors: Nurse
   - Functionality: Administer medications, update patient records, and monitor patient care.
5. View Health Information:
   - Actors: Patient
   - Functionality: View personal health information, test results, and treatment plans.
6. Manage System Configurations:
   - Actors: Admin
   - Functionality: Configure system settings, manage user accounts, and maintain system security.
7. Handle Appointments:
   - Actors: Receptionist
   - Functionality: Manage patient appointments, handle check-ins, and direct patients to the appropriate departments.

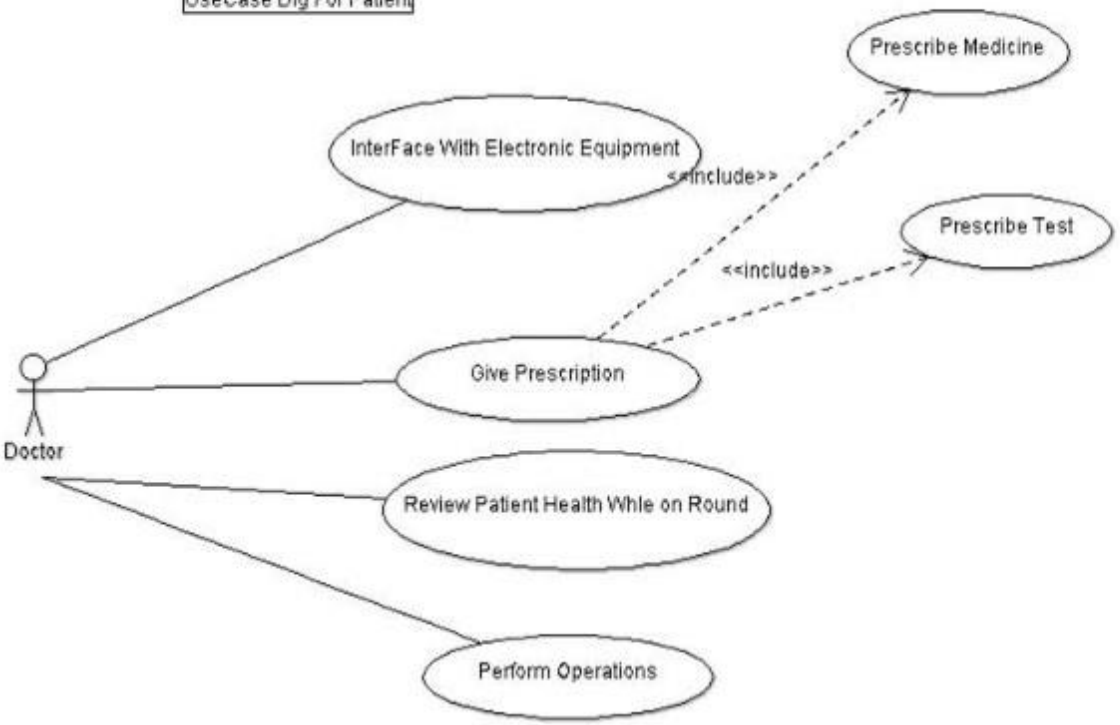
Associations:

- Doctor-Patient: The Doctor uses the system to access and update patient information.
- Nurse-Patient: The Nurse uses the system to manage patient care and update records.
- Receptionist-Patient: The Receptionist uses the system to schedule appointments and manage patient check-ins.
- Admin-System: The Admin manages system configurations and user accounts.

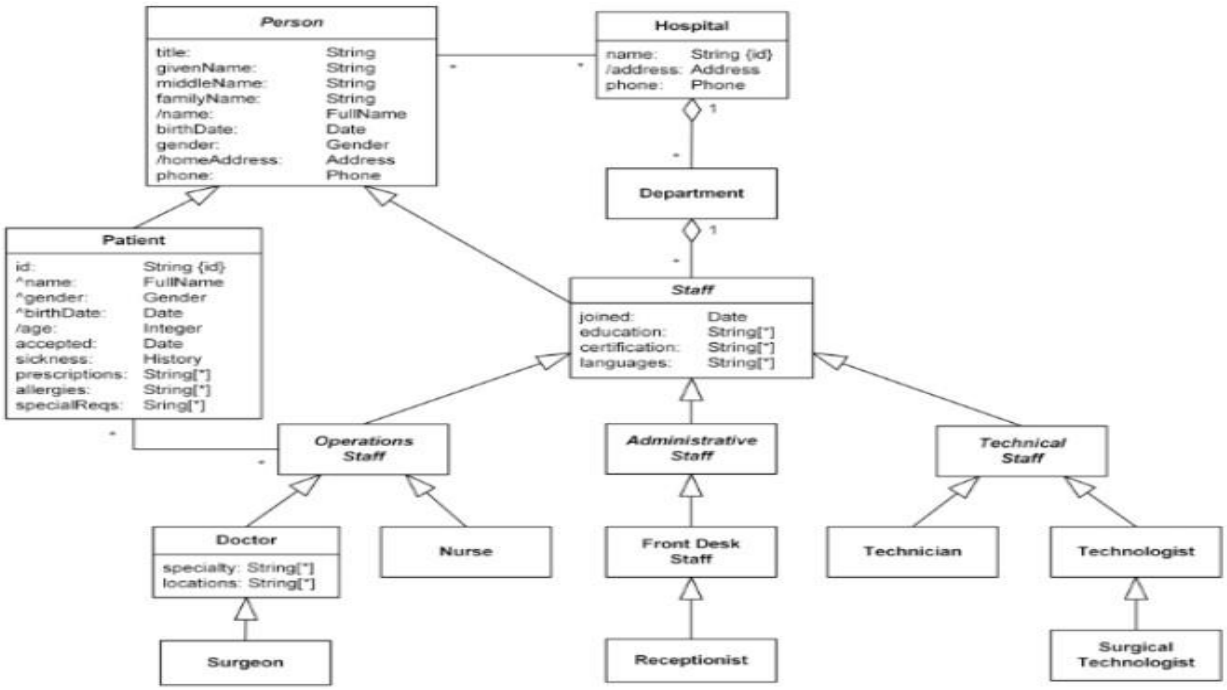
This is a simplified example, and in a real-world scenario, the use case diagram could include additional actors, use cases, and associations. Use case diagrams serve as a high-level view of system functionality and user interactions in a Hospital Management System.



UseCase Dig For Patient



3.3.2CLASS DIAGRAM:



1. System Requirements:

Functional Requirements: Identify and define the functional capabilities of the e-voting system,including voter registration, ballot creation, vote casting, and results tabulation.

Non-functional Requirements: Specify non-functional aspects such as system performance,security, usability, and reliability.

2. User Roles and Permissions:

Identify User Roles: Define different user roles within the system, such as voters, electionadministrators, and system administrators.

Permissions: Clearly outline the permissions and access levels associated with each user role toensure proper segregation of duties.

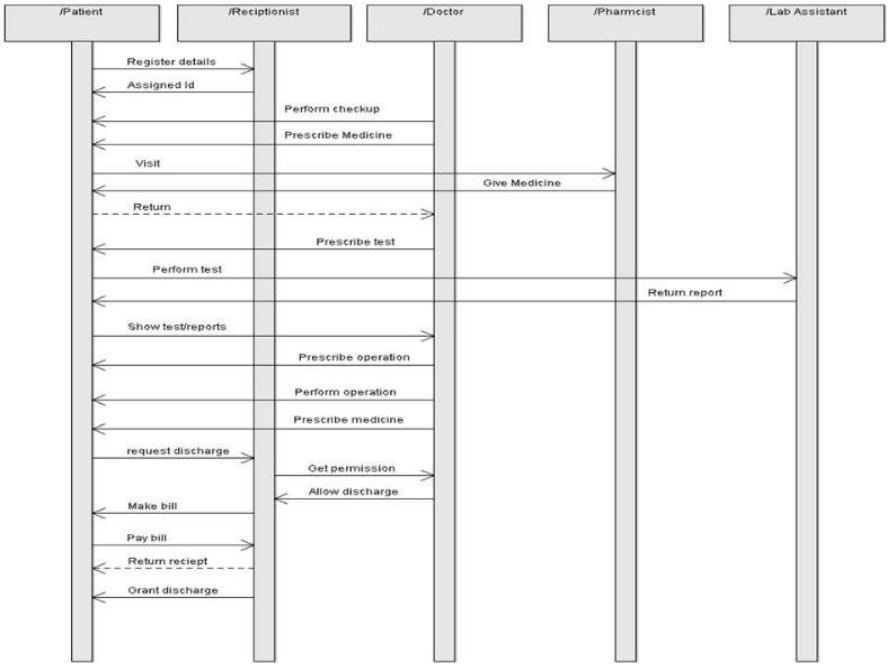
3. Security Measures:

Authentication and Authorization: Specify the mechanisms for user authentication andauthorization to prevent unauthorized access.

Encryption: Define the encryption protocols to secure data transmission and storage, includingvotes and voter information.

Audit Trails: Implement mechanisms for logging and auditing system activities to facilitateforensic analysis in case of security incidents.

3.3.3Sequence Diagram



CHAPTER-4

TECHNOLOGY DESCRIPTION

4.TECHNOLOGY DESCRIPTION

4.1 Introduction to HTML:

HTML, or Hyper-Text Markup Language, is the standard markup language used for creating and structuring content on the web. Developed by the World Wide Web Consortium (W3C), HTML uses a system of tags to define elements such as headings, paragraphs, links, images, and more. It forms the foundation of web development, providing a structured and semantic way to present information on websites. HTML works in conjunction with other web technologies like CSS (Cascading Style Sheets) and JavaScript to enhance the design and functionality of web pages. Understanding HTML is essential for anyone involved in creating and maintaining web content.

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

Features of HTML:

- It is easy to learn and easy to use.
- It is platform-independent.
- Images, videos, and audio can be added to a web page.
- Hypertext can be added to the text.
- It is a markup language.

4.2 Introduction to CSS:

Cascading **S**tyle **S**heets, fondly referred to as **CSS**, is a simply designed language intended to simplify the process of making web pages presentable. CSS allows you to apply styles to web pages. More importantly, CSS enables you to do this independently of the HTML that makes up each web page. It describes how a webpage should look: it prescribescolours, fonts, spacing, and much more. In short, you can make your website look however you want. CSS lets developers and designers define how it behaves, including how elements are positioned in the browser.

- While HTML uses tags, CSS uses rulesets. CSS is easy to learn and understand, butit provides powerful control over the presentation of an HTML document. **CSS saves time:** You can write CSS once and reuse the same sheet in multipleHTML pages.
- **Easy Maintenance:** To make a global change simply change the style, and all elements in all the webpages will be updated automatically.
- **Search Engines:** CSS is considered a clean coding technique, which means search engines won't have to struggle to "read" its content.
- **Superior styles to HTML:** CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison toHTML attributes.
- **Offline Browsing:** CSS can store web applications locally with the help of an offline cache. Using this we can view offline websites.
- **Features:**
- First and foremost, It's **FREE**.

Base: Elements like <p>, <h1…h6>, topography etc.

- **Grids:** Responsive grids.
- **Forms:** Multi-columns, stacked forms, aligned forms, grouped inputs, etc.
- **Tables:** Base tables with borders, stripping, etc.
- **Buttons**: Button effects like focus, hover, active, etc.
- **Menus:** Dropdowns, scrollable menus, vertical menus, etc.

4.3 Introduction to JavaScript:

It is well-known for the development of web pages, and many non-browser environments also use it. JavaScript is a **weakly typed language (dynamically typed)**. JavaScript can be used for **Client-side** developments as well as **Server-side** developments. JavaScript is both an imperative and declarative type of language. JavaScript contains a standard library of objects, like **Array**, **Date**, and **Math**, and a core set of language elements like **operators**, **control structures**, and **statements**.

- **Client-side:** It supplies objects to control a browser and its Document Object Model (DOM). Like if client-side extensions allow an application to place elements on an HTML form and respond to user events such as **mouse clicks**, **form input**, and **page navigation**. Useful libraries for the client side are **AngularJS**, **ReactJS** and so many others.
- **Server-side:** It supplies objects relevant to running JavaScript on a server. For if the server-side extensions allow an application to communicate with a database, and provide continuity of information from one invocation to another of the application, or perform file manipulations on a server. The useful framework which is the most famous these days is **node.js**.
- **Imperative language –** In this type of language we are mostly concerned about how it is to be done. It simply controls the flow of computation. The procedural programming approach, object, oriented approach comes under this as async await we are thinking about what is to be done further after the async call.
- **Declarative programming –** In this type of language we are concerned about howit is to be one, basically here logical computation requires. Her main goal is to describe the desired result without direct dictation on how to get it as the arrow function does.

4.4 Introduction to Django:

Django is a high-level, open-source web framework written in Python that encourages rapid development and clean, pragmatic design. Developed by the Django SoftwareFoundation, Django follows the Model-View-Controller (MVC) architectural pattern, emphasizing the "Don't Repeat Yourself" (DRY) and "Convention Over Configuration" principles.

Key features of Django include an Object-Relational Mapping (ORM) system for database interaction, a powerful templating engine, and a built-in administrative interface. It promotes the creation of robust, maintainable web applications by providing pre-built components and a structured approach to application development.

Django's "batteries-included" philosophy means it comes with a set of built-in features for common tasks, such as authentication, URL routing, form handling, and more. This allows developers to focus on building specific application features rather than dealing with routine infrastructure concerns.

Additionally, Django supports modularity and extensibility, allowing developers to integrate third-party packages and libraries easily. With a strong emphasis on security, Django provides

protections against common web vulnerabilities, making it a popular choice for building scalable and secure web applications. Whether used for prototyping or large-scale production applications, Django simplifies the development process and encourages best practices in web development.

Features:

- MVC Architecture
- Object-Relational Mapping (ORM)
- Admin Interface
- "Batteries-Included" Philosophy
- Scalability and Reusability

4.5 Introduction to Python:

Python is a versatile, high-level programming language known for its simplicity, readability, and vast ecosystem of libraries and frameworks. In the context of an Automobile ManagementSystem, Python serves as an excellent choice for various reasons:

- **Readability and Ease of Learning:**
- Python's syntax is clear and readable, making it an ideal language for developers of different expertise levels. This characteristic enhances collaboration among team members working on the Automobile Management System.
- **Rapid Development:**
- Python is renowned for its rapid development capabilities. Its concise and expressive syntax allows developers to write fewer lines of code while achieving powerful functionality. This can expedite the development of features within the Automobile Management System.

- **Extensive Libraries:**
- Python boasts a rich ecosystem of libraries and frameworks that can be seamlessly integrated into the Automobile Management System. For example, frameworks like Django can accelerate web development, while libraries such as Pandas and NumPy facilitate data processing and analysis.
- **Compatibility with Other Technologies:**
- Python is easily integrable with other technologies commonly used in the automotive industry. Whether interacting with databases, connecting to hardware components, or utilizing communication protocols, Python offers compatibility and flexibility.
- **Community Support:**
- Python has a vibrant and active community of developers worldwide. This communitysupport ensures that developers working on the Automobile Management System can easily find solutions to challenges, access documentation, and benefit from shared knowledge.
- **Scalability:**
- Python's scalability is well-demonstrated in various domains, from small scripts to large-scale applications. As the requirements of the Automobile Management Systemevolve, Python provides the scalability needed for future enhancements.
- **Cross-Platform Compatibility:**
- Python is a cross-platform language, meaning code developed on one platform can runon various operating systems. This ensures that the Automobile Management System can be deployed on different environments without significant modifications.
- **Data Processing and Analysis:**
- For systems that involve handling large datasets, Python excels in data processing andanalysis. Utilizing libraries like Pandas and NumPy, developers can efficiently manage

- and derive insights from automotive data within the system.

In summary, Python's readability, rapid development capabilities, extensive libraries, compatibility with other technologies, community support, scalability, cross-platform compatibility, and prowess in data processing make it a compelling choice for developing and maintaining an efficient and effective Automobile Management System.

4.6 FEATURES:

- **Interpreted**
- There are no separate compilation and execution steps like C and C++.
- Directly *run* the program from the source code.
- Internally, Python converts the source code into an intermediate form called bytecodes which is then translated into native language of specific computer to run it.
- No need to worry about linking and loading with libraries, etc.
- **Platform Independent**
- Python programs can be developed and executed on multiple operating systemplatforms.
- Python can be used on Linux, Windows, Macintosh, Solaris and many more.
- **Free and Open Source;** Redistributable
- **High-level Language**
- In Python, no need to take care about low-level details such as managing the memoryused by the program.

CHAPTER-5
SAMPLE CODE

5.SAMPLE CODE

1. Login.html: (Displays Login Page)

```
  {% extends 'navigation.html' %}
  {% load static %}
  {% block body %}

<html lang="en" dir="ltr">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
 <title>Admin Login </title>

 <style type="text/css">
  body {
   color: #aa082e;
   background-color: #b6bde7;
   font-family: 'Roboto', sans-serif;
  }

  a:link {
   text-decoration: none;
  }

  .note {
   text-align: center;
   height: 80px;
   background: -webkit-linear-gradient(left, #0072ff, #8811c5);
   color: #fff;
   font-weight: bold;
   line-height: 80px;
  }

  .form-content {
   padding: 4%;
   border: 1px solid #ced4da;
   margin-bottom: 1%;
   text-align : center;
  }

  .form-control {
   border-radius: 1.5rem;
   text-align : center;
  }

  .btnSubmit {
   border: none;
   border-radius: 1.5rem;
   padding: 1%;
   width: 30%;
cursor: pointer;
background: #0062cc;
```

```
color: #fff;
}
</style>
</head>
<body>

{% if error == "no" %}
<script>
alert('Logged In Successfully');
window.location=('{% url 'admin_home' %}');
</script>
{% endif %}

{% if error == "yes" %}
<script>
alert('Something went wrong, Try Again');
</script>
{% endif %}

<br><br><br><br>
<form method="post">
{% csrf_token %}
<div class="container register-form mt-2" style="width : 600px;">
<div class="form">
<div class="note">
<p>Admin Login</p>
</div>

<div class="form-content">
<div class="row mt-5">

<div class="col-md-12">
<div class="form-group">
<input type="text" name="uname" class="form-control py-3" placeholder="Enter User Name">
</div>
</div>

<div class="col-md-12">
<div class="form-group">
<input type="password" name="pwd" class="form-control py-3" placeholder="Enter Password"><br>
</div>
</div>

</div>
<button type="submit" class="btnSubmit">Login</button>
</div>
</div>
</div>

</form>
```

2.Contact.html:(Display contact Page)

```
{% extends 'navigation.html' %}
{% load static %}
{% block body %}


{% if error == "no" %}
<script>
alert('Your Message has been Send Successfully');
window.location=('{% url 'contact' %}');
</script>
{% endif %}

{% if error == "yes" %}
<script>
alert('Something went wrong, Try Again');
</script>
{% endif %}


<section style="background-color : #21446B; height : 650px;">
<div class="container py-5 mt-3">
<div class="card">
<div class="card-body">
<h1 class="font-weight-light text-center py-3">Contact Us</h1>
<div class="row">
<div class="col-lg-6 col-md-12 col-sm-12">

<div class="row pt-3">
<div class="col-lg-1 offset-1 col-md-2 col-sm-2">
<span style="font-size: 30px; color: cadetblue"><i class="fas fa-map-marker-alt"></i></span>
</div>

<div class="col-lg-10 col-md-9 col-sm-9">
<h3 class="font-weight-light">Find Us at the office</h3>
<p>232 Sweet Homes<br>
B Sector, Indrapuri<br>
Bhopal(M.P.)
</p>
</div>
</div>

<div class="row pt-3">
<div class="col-lg-1 offset-1 col-md-2 col-sm-2">
<span style="font-size: 30px; color: coral"><i class="fas fa-phone-volume"></i></span>
</div>

<div class="col-lg-10 col-md-9 col-sm-9">
<h3 class="font-weight-light">Give Us a Ring</h3>
<p>Vara prasad<br>
+91 8602768216<br>
kakinada
</p>
</div>
</div>

</div>
```

```
<div class="col-lg-6 col-md-12 col-sm-12">
<form method="post">
{% csrf_token %}
<div class="form-row">
<div class="form-group col-lg-6 col-md-12 col-sm-12">
<label>Full Name</label>
<input type="text" name="name" class="form-control" placeholder="Full Name" required>
</div>

<div class="form-group col-lg-6 col-md-12 col-sm-12">
<label>Contact Number</label>
<input type="text" name="contact" class="form-control" placeholder="Contact Number" required>
</div>

<div class="form-group col-lg-6 col-md-12 col-sm-12">
<label>Email Id</label>
<input type="email" name="email" class="form-control" placeholder="Email ID" required>
</div>

<div class="form-group col-lg-6 col-md-12 col-sm-12">
<label>Subject</label>
<input type="text" name="subject" class="form-control" placeholder="Enter Subject" required>
</div>

<div class="form-group col-lg-12 col-md-12 col-sm-12">
<label>Message</label>
<textarea name="message" class="form-control" placeholder="Message....."></textarea>
</div>

<div class="form-group col-lg-6 col-md-12 col-sm-12">
<input type="submit" value="Send" class="form-control btn btn-primary">
</div>

<div class="form-group col-lg-6 col-md-12 col-sm-12">
<input type="reset" value="Clear" class="form-control btn btn-primary">
</div>

</div>
</form>
</div>
</div>

</div>
</div>
</div>

</section>

{% endblock%}
```

3.index.html:(Display List of the index page)

```
{% extends 'navigation.html' %}
{% load static %}
{% block body %}

<head>
  <style media="screen">
    .jumbotron {
      width : 100;
      height : 550px;
      background-image: url('{% static "images/h3.jpg" %}');
      background-size: cover;
      background-repeat: no-repeat;
    }

    .jumbotron h5,
    h3 {
      text-align: center;
    }


    .alert {
      margin: 0px;
    }

    .glow {
      font-size: 50px;
      color: black;
      margin-right : 5px;
      text-align: center;
      -webkit-animation: glow 1s ease-in-out infinite alternate;
      -moz-animation: glow 1s ease-in-out infinite alternate;
      animation: glow 1s ease-in-out infinite alternate;
    }

    @-webkit-keyframes glow {
      from {
        text-shadow: 0 0 10px #eeeeee, 0 0 20px #000000, 0 0 30px #000000, 0 0 40px #000000, 0 0 50px
#9554b3, 0 0 60px #9554b3, 0 0 70px #9554b3;
      }
      to {
        text-shadow: 0 0 20px #eeeeee, 0 0 30px #ff4da6, 0 0 40px #ff4da6, 0 0 50px #ff4da6, 0 0 60px
#ff4da6, 0 0 70px #ff4da6, 0 0 80px #ff4da6;
      }
    }
  </style>
</head>
<br>
<br>

<div class="jumbotron" style="margin-bottom: 0px;margin-top: 0px;">
  <br><br>
  <h5 class="display-3 glow">"A hospital alone shows what war is."</h5>
  <br><br><br><br>
</div>{% endblock%}
```

4.About.html:(Displays About page)

```
    {% extends 'navigation.html' %}
{% load static %}
{% block body %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">

<section id="about">
<div class="container mt-5">
<div class="row">
<div class="col-sm-6">
<div class="section-title">
<div class="section-subtitle">About Us</div>
<h2 class="section-main-title">
All About <strong>Hospitals</strong>
</h2>
</div>
<div class="about-item"><p>A hospital is a place where a person goes to be healed when he she is sick or injured. Hospitals also treat people who do not stay overnight, called outpatients. Doctors and nurses work at hospitals.Doctors make use of advanced medical technology to heal patients.</p>
</div>
</div>

<div class="col-sm-4 offset-sm-2">
<div class="about-box">
 <img src="{% static 'images/about-1.jpg' %}">
</div>
</div
   </div>

<div class="row mt-5">
<div class="col-sm-4">
<div class="about-boxx">
  <img src="{% static 'images/about1.jpg' %}">
</div>
</div>
<div class="col-sm-6 offset-sm-2">
<div class="about-item mt-5">
<h4><span style="color:red">All About</span> <span style="color:green"><b>Doctor & Nourse</b></span></h4><p>Doctors and Nurses may refer to a children's game of imaginative role-playing where medical profession roles are adopted and acted out.A nurse is a person who is trained to give care to people who are sick or injured.Nurses work with doctors and other health care workers to make patients.</p>
</div>
</div>
</div>
  <div class="row mt-5">
<div class="col-sm-6">
<div class="about-item mt-5">
<h4><span style="color:green">All About</span> <span style="color:blue"><b>Sweeper</b></span></h4>
<p>A Hospital cleaner a cleaning operative is a type of industrial ordomestic worker who cleans homes,commercial premises and also clean
the Hospital Plateform.A cleaner is someone who takes away garbage and cleans surfaces. Cleaners
```

```
sometimes repair things,and maintain their equipment in good working.</p>
                                                    </div>
</div>

<div class="col-sm-4 offset-sm-2">
<div class="about-box">
<img src="{% static 'images/about2.jpg' %}">
 </div>
</div>
</div>
</div></section>

{% endblock%}
```

5.Views.py

```
from django.shortcuts import render,redirect
from django.contrib.auth.models import User
from django.contrib.auth import authenticate,logout,login
from .models import *
from datetime import date

# Create your views here.

def About(request):
    return render(request,'about.html')

def Index(request):
    return render(request,'index.html')

def contact(request):
    error = ""
    if request.method == 'POST':
        n = request.POST['name']
        c = request.POST['contact']
        e = request.POST['email']
        s = request.POST['subject']
        m = request.POST['message']
        try:
            Contact.objects.create(name=n, contact=c, email=e, subject=s, message=m,
msgdate=date.today(), isread="no")
            error = "no"
        except:
            error = "yes"
    return render(request, 'contact.html', locals())

def adminlogin(request):
    error = ""
    if request.method == 'POST':
        u = request.POST['uname']
        p = request.POST['pwd']
        user = authenticate(username=u, password=p)
        try:
            if user.is_staff:
                login(request, user)
                error = "no"
            else:
                error = "yes"
        except:
            error = "yes"
    return render(request,'login.html', locals())

def admin_home(request):
    if not request.user.is_staff:
        return redirect('login_admin')
    dc = Doctor.objects.all().count()
    pc = Patient.objects.all().count()
    ac = Appointment.objects.all().count()

    d = {'dc': dc, 'pc': pc, 'ac': ac}
    return render(request,'admin_home.html', d)
```

```
def Logout(request):
    logout(request)
    return redirect('index')

def add_doctor(request):
    error=""
    if not request.user.is_staff:
        return redirect('login')
    if request.method=='POST':
        n = request.POST['name']
        m = request.POST['mobile']
        sp = request.POST['special']
        try:
            Doctor.objects.create(name=n,mobile=m,special=sp)
            error="no"
        except:
            error="yes"
    return render(request,'add_doctor.html', locals())

def view_doctor(request):
    if not request.user.is_staff:
        return redirect('login')
    doc = Doctor.objects.all()
    d = {'doc':doc}
    return render(request,'view_doctor.html', d)

def Delete_Doctor(request,pid):
    if not request.user.is_staff:
        return redirect('login')
    doctor = Doctor.objects.get(id=pid)
    doctor.delete()
    return redirect('view_doctor.html')

def edit_doctor(request,pid):
    error = ""
        • if not request.user.is_authenticated:
        return redirect('login')
    user = request.user
    doctor = Doctor.objects.get(id=pid)
    if request.method == "POST":
        n1 = request.POST['name']
        m1 = request.POST['mobile']
        s1 = request.POST['special']

        doctor.name = n1
        doctor.mobile = m1
        doctor.special = s1

        try:
            doctor.save()
            error = "no"
        except:
            error = "yes"
    return render(request, 'edit_doctor.html', locals())

def add_patient(request):
```

```
    error = ""
    if not request.user.is_staff:
        return redirect('login')
    if request.method == 'POST':
        n = request.POST['name']
        g
        m = request.POST['mobile']
        a = request.POST['address']
        try:
            Patient.objects.create(name=n, gender=g, mobile=m, address=a)
            error = "no"
        except:
            error = "yes"
    return render(request,'add_patient.html', locals())

def view_patient(request):
    if not request.user.is_staff:
        return redirect('login')
    pat = Patient.objects.all()
    d = {'pat':pat}
    return render(request,'view_patient.html', d)

def Delete_Patient(request,pid):
    if not request.user.is_staff:
        return redirect('login')
    patient = Patient.objects.get(id=pid)
    patient.delete()
    return redirect('view_patient.html')

def edit_patient(request,pid):
    error = ""
    if not request.user.is_authenticated:
        return redirect('login')
    user = request.user
    patient = Patient.objects.get(id=pid)
    if request.method == "POST":
        n1 = request.POST['name']
        m1 = request.POST['mobile']
        g1 = request.POST['gender']
        a1 = request.POST['address']

        patient.name = n1
        patient.mobile = m1
        patient.gender = g1
        patient.address = a1
        try:
            patient.save()
            error = "no"
        except:
            error = "yes"
    return render(request, 'edit_patient.html', locals())



def add_appointment(request):
    error=""
    if not request.user.is_staff:
```

```
        return redirect('login')
    doctor1 = Doctor.objects.all()
    patient1 = Patient.objects.all()
    if request.method=='POST':
        d = request.POST['doctor']
        p = request.POST['patient']
        d1 = request.POST['date']
        t = request.POST['time']
        doctor = Doctor.objects.filter(name=d).first()
        patient = Patient.objects.filter(name=p).first()
        try:
            Appointment.objects.create(doctor=doctor, patient=patient, date1=d1, time1=t)
            error="no"
        except:
            error="yes"
    d = {'doctor':doctor1,'patient':patient1,'error':error}
    return render(request,'add_appointment.html', d)

def view_appointment(request):
    if not request.user.is_staff:
        return redirect('login')
    appointment = Appointment.objects.all()
    d = {'appointment':appointment}
    return render(request,'view_appointment.html', d)

def Delete_Appointment(request,pid):
    if not request.user.is_staff:
        return redirect('login')
    appointment1 = Appointment.objects.get(id=pid)
    appointment1.delete()
    return redirect('view_appointment.html')

def unread_queries(request):
    if not request.user.is_authenticated:
        return redirect('login')
    contact = Contact.objects.filter(isread="no")
    return render(request,'unread_queries.html', locals())

def read_queries(request):
    if not request.user.is_authenticated:
        return redirect('login')
    contact = Contact.objects.filter(isread="yes")
    return render(request,'read_queries.html', locals())

def view_queries(request,pid):
    if not request.user.is_authenticated:
        return redirect('login')
    contact = Contact.objects.get(id=pid)
    contact.isread = "yes"
    contact.save()
    return render(request,'view_queries.html', locals())
```

6.Urls.py:

```
from django.contrib import admin
from django.urls import path
from hospitals.views import *

urlpatterns = [
    path('admin/', admin.site.urls),
    path('about/', About, name="about"),
    path('',Index,name='index'),
    path('contact', contact, name='contact'),
    path('login', adminlogin, name='login'),
    path('admin_home', admin_home, name='admin_home'),
    path('logout', Logout, name='logout'),
    path('add_doctor', add_doctor, name='add_doctor'),
    path('view_doctor', view_doctor, name='view_doctor'),
    path('delete_doctor/<int:pid>', Delete_Doctor, name='delete_doctor'),
    path('add_patient', add_patient, name='add_patient'),
    path('view_patient', view_patient, name='view_patient'),
    path('delete_patient/<int:pid>', Delete_Patient, name='delete_patient'),
    path('add_appointment', add_appointment, name='add_appointment'),
    path('view_appointment', view_appointment, name='view_appointment'),
    path('delete_appointment/<int:pid>', Delete_Appointment, name='delete_appointment'),
    path('edit_doctor/<int:pid>',edit_doctor,name='edit_doctor'),
    path('edit_patient/<int:pid>',edit_patient,name='edit_patient'),
    path('unread_queries', unread_queries, name='unread_queries'),
    path('read_queries', read_queries, name='read_queries'),
    path('view_queries/<int:pid>', view_queries, name='view_queries'),

]
```

CHAPTER-6

TESTING

6.TESTING

6.1 Introduction

Testing is a critical phase in the software development life cycle that involves systematically evaluating a system or application to identify and rectify defects, ensuring its reliability, functionality, and performance. The primary goal of testing is to deliver high-quality software that meets user expectations and adheres to specified requirements. Testing encompasses various activities, methodologies, and levels to verify and validate different aspects of the software. A fault will become a failure if the precise computation conditions are met, one in all them being that the faulty portion of computer software executes on the CPU. Afault can even be converted into a failure when the software is ported to a unique hardware platform or a unique compiler, or when the software gets extended. Software testing is that thetechnical investigation of the merchandise under test to supply stakeholders with quality relatedinformation.

6.2 System Testing and Implementation:

The purpose is to exercise the various parts of the module code to detect coding errors. After this the modules are gradually integrated into subsystems, which are then integrated themselves too eventually forming the whole system. During integration of module integrationtesting is performed. The goal of this is often to detect designing errors, while focusing the interconnection between modules. After the system was put together, system testing is performed. Here the system is tested against the system requirements to determine if all requirements were met and also the system performs as specified by the wants. Finally acceptingtesting is performed to demonstrate to the client for the operationof the system.

For the testing to achieve success, proper selection of the test suit is crucial. There are two different approaches for choosing action. The software or the module to be tested is treatedas a recorder, and therefore the test cases are decided supported the specifications of the systemor module. For this reason, this type of testing is additionally called “black box testing”. After this the modules are gradually integrated into subsystems, which are then integrated themselvestoo eventually forming the whole system. During integration of module integration testing is performed. The goal of this is often to detect designing errors, while focusing the interconnection between modules. After the system was put together, system testing is performed. Here the system is tested against the system requirements to determine if all requirements were met and also the system performs as specified by the wants.

The focus here is on testing the external behaviour of the system. In structural testing the test cases are decided supported the logic of the module to be tested. a typical approach hereis to realize some sort of coverage of the statements within the code. the 2 styles of testing are complementary: one tests the external behaviour, the opposite tests the inner structure. Testingis a particularly critical and time-consuming activity. It requires proper planning of the testing process. Frequently Courier Management System 60 the testing process starts with the test plan.This plan identifies all testing related activities that

After this the modules are gradually integrated into subsystems, which are then integrated themselves too eventually forming the whole system. During integration of module integration testing is performed. The goal of this is often to detect designing errors, while focusing the interconnection between modules. After the system was put together, system testing is performed. Here the system is tested against the system requirements to determine if all requirements were met and also the system performs as specified by the wants. Finally accepting testing is performed to demonstrate to the client for the operationof the system.

For the testing to achieve success, proper selection of the test suit is crucial. There are two different approaches for choosing action. The software or the module to be testedis treated as a recorder, and therefore the test cases are decided supported the specifications of the system or module. For this reason, this type of testing is additionally called “black box testing”. After this the modules are gradually integrated into subsystems, which are then integrated themselves too eventually forming the whole system. During integration of module integration testing is performed. The goal of this is often to detect designing errors, while focusing the interconnection between modules. has got to be performed and specifies the schedule,allocates the resources, and specifies guidelines for testing. The test plan specifies conditions that oughtto be tested; different units to be tested, and also the manner within which the module are integrated together. Then for various test unit, a test suit specification document is produced, which lists all the various test cases, along with the expected outputs, that may be used for testing. During the testing of the unit the required test cases are executed and also the actual results arecompared with the expected outputs. the ultimate output of the testing phase is that the testing report and also the error report, or a group of such reports. Each test report contains acollection of test cases and therefore the results of executing the code with the test cases.The error report describes the errors encountered and therefore the action taken to get rid of the error.

6.3 Testing Techniques

Testing may be a process, which reveals errors within the program. it's the foremost quality measure employed during software development. During testing, the program is executed with a collection of conditions referred to as test cases and therefore the output is evaluated to see whether the program is performing needless to say. so as to form sure that the system doesn't have errors, the various levels of testing strategies that are applied at differing phases of software development are:

Black Box Testing

In this strategy some test cases are generated as input conditions that fully execute all functional requirements for the program. This testing has been uses to find errors in the following categories:

a. Incorrect or missing functions
b. Interface errors
c. Errors in data structure or external database access
d. Performance errors
e. Initialization and termination errors.
f. In this testing only the output is checked for correctness. The logical flow of the datais notchecked.

White Box Testing:

In this testing, the test cases are generated on the logic of each module by drawing flowgraphs of that module and logical decisions are tested on all the cases. It has been uses to generate the test cases in the following cases.Guarantee that all independent paths have been executed.

- Execute all logical decisions on their true and false sides.
- Execute all loops at their boundaries and within their operational.
- Execute internal data structures to ensure their validity.

6.4 Testing Strategies:

Unit testing:

Unit testing involves the look of test cases that validate that the inner program logic is functioning properly, which program inputs produce valid outputs. All decision branches andinternal code flow should be validated. it's the testing of individual software units of the appliance .it is done after the completion of a personal unit before integration.

structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a selected business process, application, and/or system configuration. Unit tests make sure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

This System consists of 3 modules. Those are Reputation module, route discovery module, audit module. Each module is taken as unit and tested. Identified errors arecorrected and executable unit are obtained.

Integration testing:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of componentsis correct and consistent. Integration testing is specifically aimed at exposing the problems thatarise from the combination of components.

System Testing:

System testing ensures that the entire integrated software system meets requirements. It testsa configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

Functional Testing:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items

Valid Input : identified classes of valid input must be accepted. Invalid Input : identified classes of invalid input must be rejected.Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Businessprocess flows; data fields, predefined processes, and successive processes.

development of an event management system. It helps identify defects, ensures the system functions as expected, and provides confidence in its reliability and quality. Here are some types of testing that can be performed for an event management system:

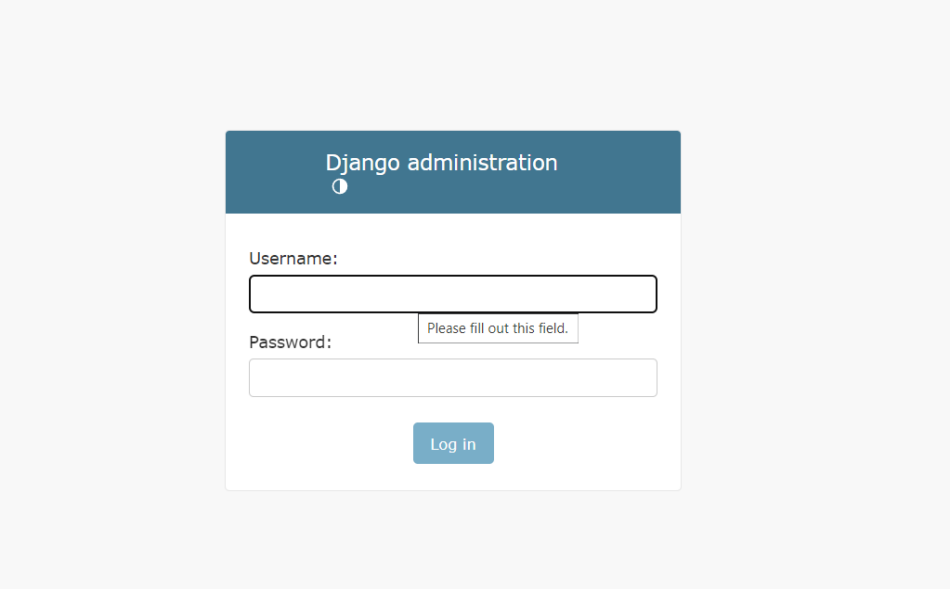
6.5 SAMPLE TEST CASES SPECIFICATIONS

Test case id	**Test case name**	**Input**	**Expected output**	**Observed output**	**Result**
T1	User login	Enter The valid id	User login should be done successfully	User login successfully	Pass
T2	User login	Enter the invalid id	User login should be done unsuccessfully	User login unsuccessfully	Pass

CHAPTER-7
SCREEN SHOTS

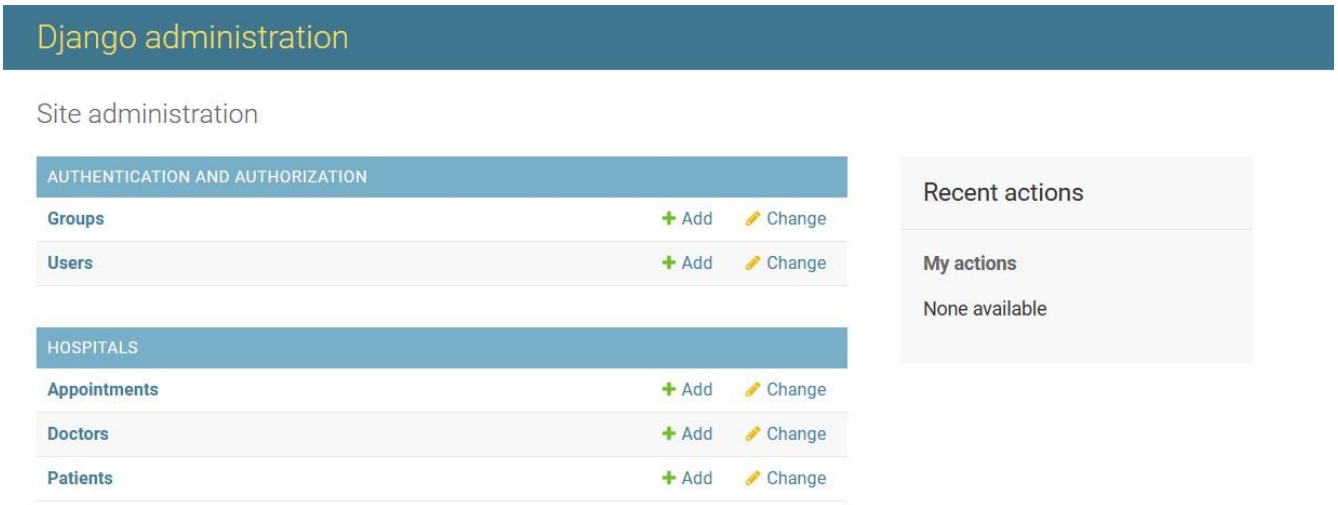
7.SCREEN SHOTS

Admin Login page:



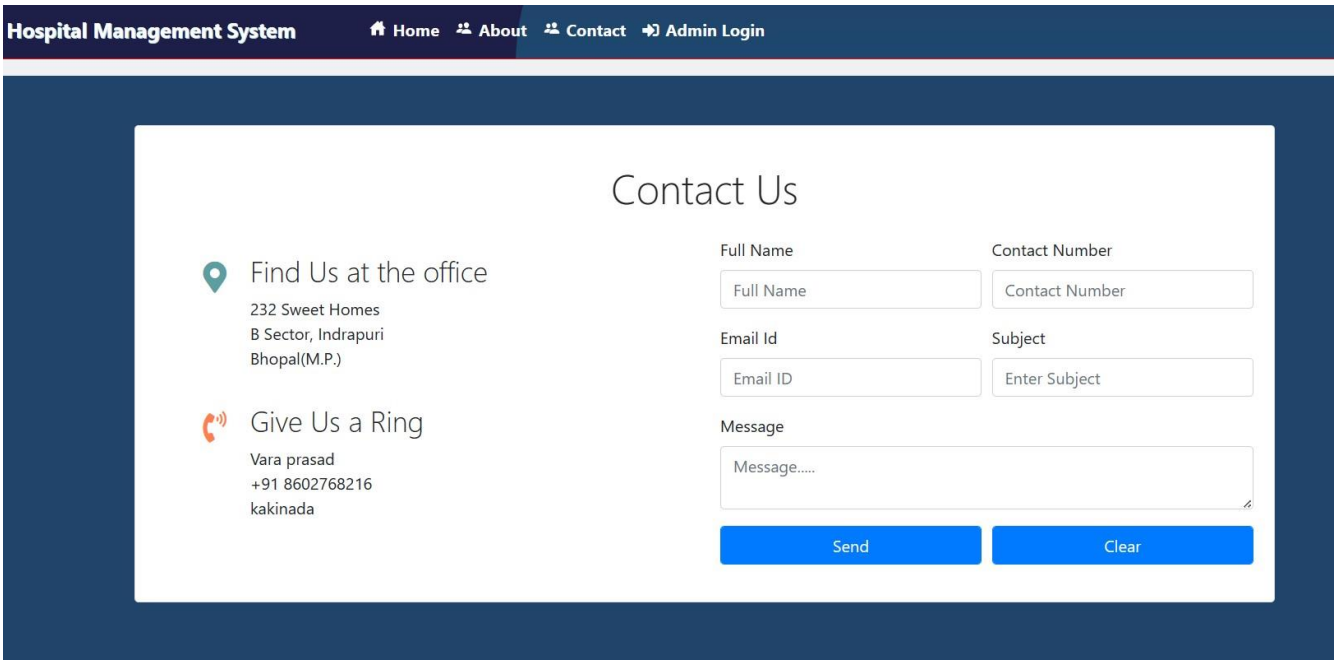
Description: The above screenshot shows the total code which is used admin/loginthe admin.

Admin page



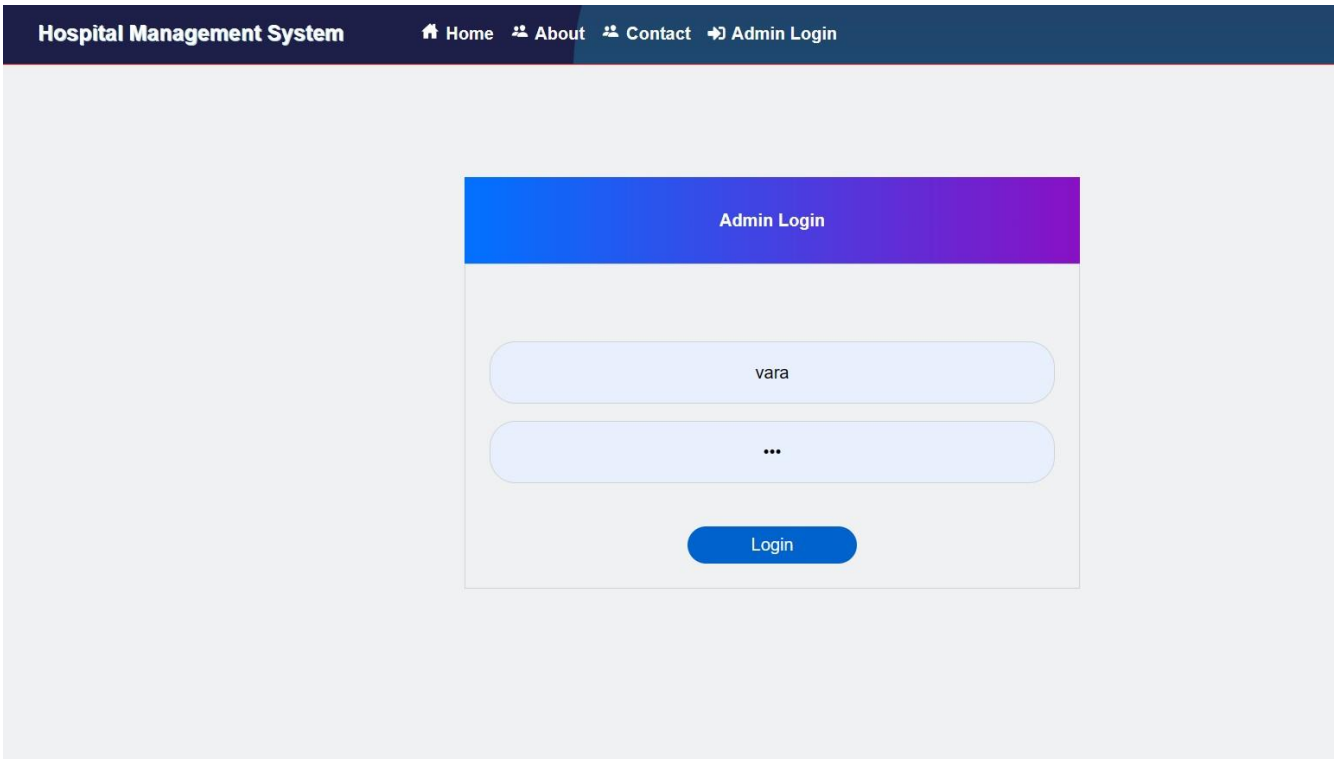
Description: The above screenshot shows the Admin page which is used toregister/inthe list.

contact Page:



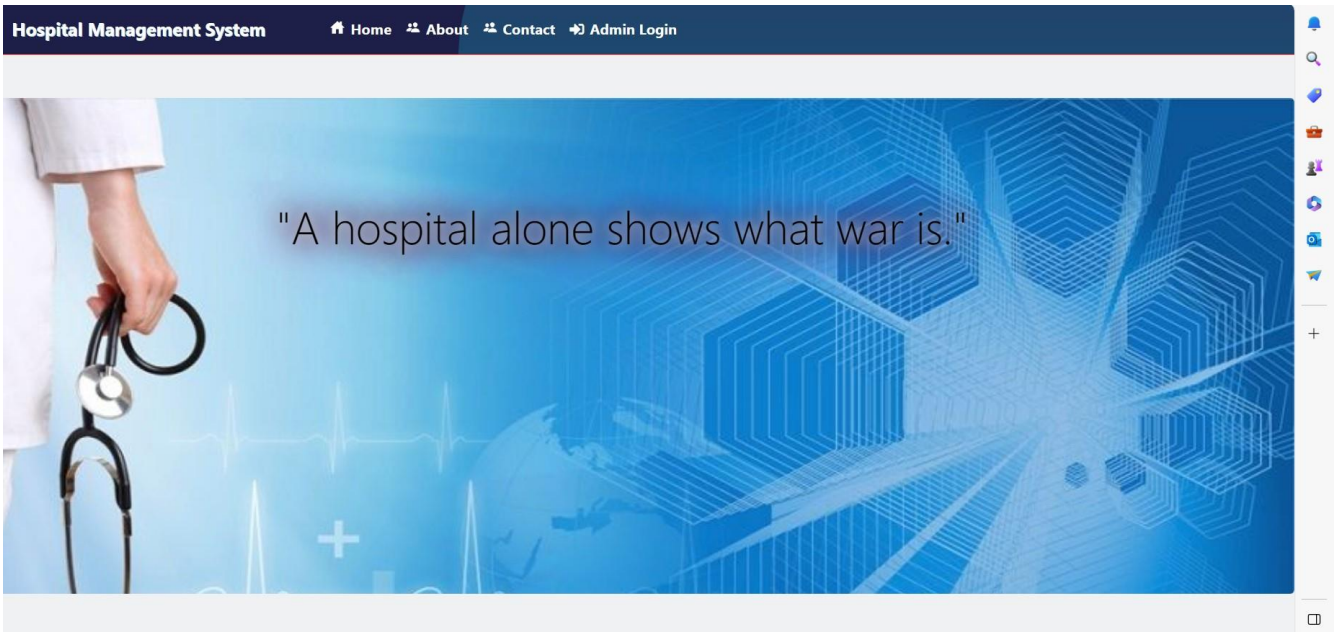
Description: The above screenshot shows the contact page which is used to signup/loginthe

Login page:



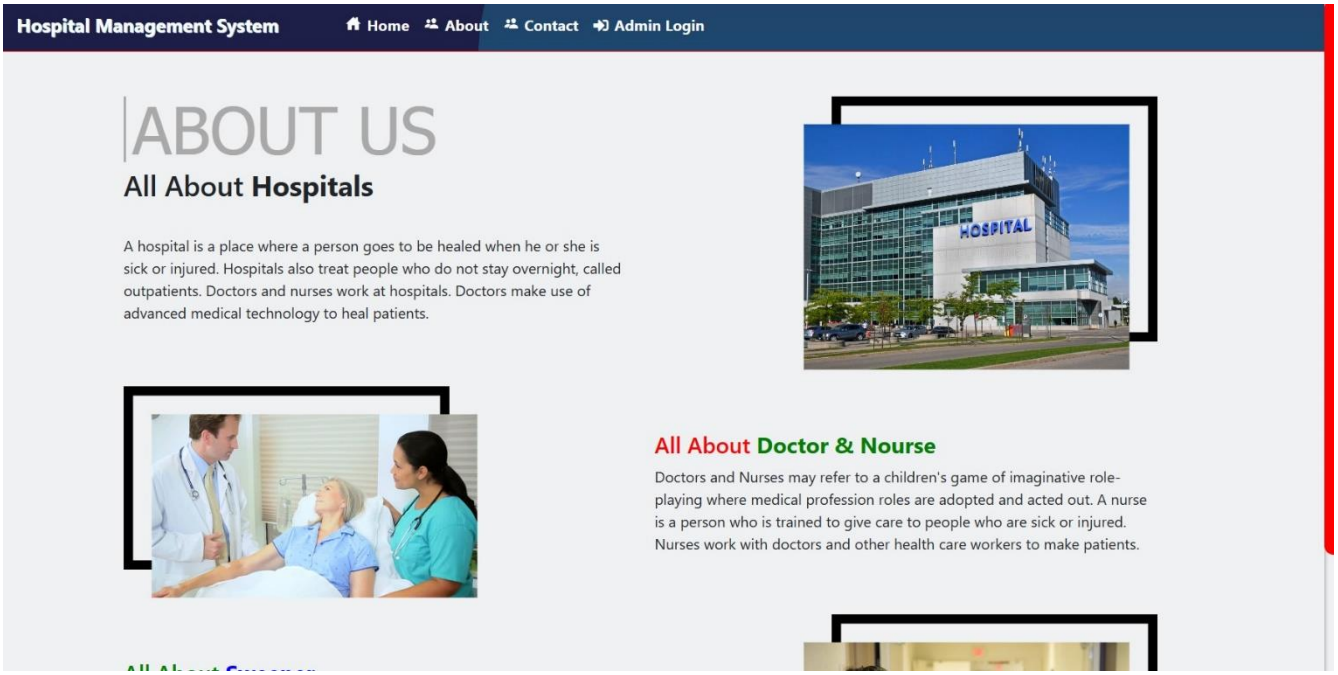
Description: The above screenshot shows the login page which is used to Display theDashboard.

Home page:



Description: The above screenshot shows the home page which is used to homepage view.

About page :



Description: The above screenshot shows the about page which is used to InventoryCategory List.

CONCLUSION

conclusion, the design and implementation of a Hospital Management System (HMS) play a crucial role in enhancing the efficiency, accuracy, and overall quality of healthcare services. The use case diagram and flowchart presented earlier provide a high-level overview of the system's functionalities, interactions, and processes.

The Hospital Management System is designed to meet the diverse needs of various stakeholders within the healthcare environment, including administrators, doctors, nurses, receptionists, and patients. By incorporating user-friendly interfaces and robust functionalities, the system aims to streamline key processes such as patient information management, appointment scheduling, diagnosis, prescription, and overall patient care.

BIBLIOGRAPHY

TEXT BOOKS:

1) Beginning Web Programing-Jon Duckett Wrox.

2) Web Technologies, Uttam K Roy, Oxford University Press.

3) HTML and CSS Quick Start Guide - David Durocher

4) Deno Web Development - Gary Cormell

WEB SITES:

1) https://www.geeksforgeeks.org/

2) http://www.wikipedia.org

3) http://www.wers.com

4) http://www.w3schools.com/fault.ap